T E C H N I C A L P R O J E C T R E P O R T

Student Performance Advisor

AI-Powered Academic Risk Assessment Expert System

Version 1.0

2025

Classification: Internal / Academic Use

Document Information

The following table summarises the key attributes and classification of this technical report.

Project Title	Student Performance Advisor
Document Type	Technical Project Documentation & System Report
Version	1.0 — Initial Release
System Type	Rule-Based Expert System (Forward Chaining)
Tech Stack	Python (Flask) · React (Vite) · Rule-Based AI
Classification	Internal / Academic

1. Executive Summary

The Student Performance Advisor is a rule-based expert system engineered to assess academic risk at the individual student level and to deliver personalised, evidence-backed recommendations. The system bridges a practical gap in academic institutions by providing counsellors, tutors, and administrators with a data-driven tool that can flag at-risk students early and suggest targeted interventions.

The solution is composed of a Python backend powered by Flask and a modern React frontend. It applies forward-chaining inference over a structured knowledge base covering attendance patterns, academic performance, behavioural indicators, and compound risk factors. Each recommendation is accompanied by a certainty factor score and a full reasoning trace, ensuring transparency and auditability.

This document provides a complete technical reference for the project, covering system architecture, data flows, rule logic, API contracts, validation rules, deployment instructions, and a roadmap for future improvements.

2. Project Overview

2.1 Purpose & Scope

Academic institutions frequently struggle to identify students who are at risk of poor performance or withdrawal until the situation has already deteriorated. The Student Performance Advisor addresses this challenge by encoding expert academic knowledge into a formal rule engine that can evaluate student profiles in real time.

The primary objectives of the system are:

- Provide an automated, explainable risk assessment for any student profile.
- Surface the key risk factors and academic strengths for each student.
- Generate prioritised, actionable recommendations for academic support.
- Expose these capabilities through both a web UI and a programmatic API.

2.2 Target Users

- Academic counsellors and student advisors
- Faculty members monitoring student engagement
- Institutional administrators tracking cohort-level risk
- Developers integrating risk assessment into existing learning management systems

2.3 Key Features

- Rule-based forward-chaining inference engine with certainty factors

- Structured knowledge base with five rule categories and conflict resolution
- Full reasoning trace returned with every analysis request
- Input validation with hard errors and informational warnings
- REST API with three endpoints and JSON contracts
- Interactive React frontend with sample profile loading
- Console demo mode and a ten-scenario automated test suite

3. System Architecture

The system is structured as a layered application comprising seven major modules. Each layer has a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces. The table below maps each layer to its directory and primary function.

Layer	Module	Responsibility
Frontend	frontend/	React + Vite UI; forms, result cards, sample profile loading
API Layer	api/	Flask server exposing health, samples, and analysis endpoints
Inference Engine	inference_engine/	Forward-chaining rule evaluator with conflict resolution
Knowledge Base	knowledge_base/	Expert rules for attendance, academics, behavior, intervention
Input Handler	input_handler/	StudentProfile model and field validation logic
Output Handler	output_handler/	Risk scoring, recommendation formatting, and report generation
Tests	tests/	10 sample student scenarios and validation runner

3.1 Frontend (frontend/)

The frontend is a single-page application built with React and the Vite build toolchain. It provides an intuitive form for entering student profile data, renders risk assessment results in structured cards, and includes a sample-profile loader for quick demonstrations. API communication is handled through the Axios library.

3.2 Backend API (api/)

The backend is a Flask application that serves as the integration point between the frontend and the inference engine. On startup it loads the knowledge base once into memory to ensure low-latency inference. Route logic is organised into blueprints under api/routes/, and environment configuration is managed via api/config.py.

3.3 Inference Engine (inference_engine/)

The inference engine implements forward chaining over a set of working-memory facts derived from the student profile. It maintains a registry of fired rules to prevent duplicate execution, employs a maximum-iteration guard against infinite loops, and uses priority-ranked conflict resolution to determine the order of rule firing. Each inference step is recorded in a reasoning trace.

3.4 Knowledge Base (knowledge_base/)

The knowledge base encapsulates the domain expertise of academic risk assessment as structured rules. Each rule is defined with a name, a set of conditions, a set of actions, a certainty factor (CF), a priority value, and a plain-language description. Rules add conclusions to working memory as they fire, allowing downstream rules to build on earlier inferences.

3.5 Input Handler (input_handler/)

This module defines the StudentProfile data model and validates every incoming profile before inference begins. Validation errors halt processing immediately; warnings are logged and returned to the caller without blocking the analysis.

3.6 Output Handler (output_handler/)

The RecommendationProcessor transforms raw engine conclusions into structured output, computing the risk score, enumerating strengths, and ranking risk factors. The ReportGenerator produces both console-formatted text for CLI use and structured JSON-like dictionaries for API responses.

4. Data Flow

The following steps describe the end-to-end processing pipeline for a single analysis request:

Step 1 Input Submission

The user submits a student profile through the React frontend or sends a POST request directly to the API.

Step 2 Payload Receipt

The Flask backend receives the JSON payload at the /api/analyze endpoint.

Step 3 Profile Creation

The payload is deserialized into a StudentProfile object and passed to the input validator. Any hard errors cause an immediate 400 response.

Step 4 Fact Loading

The validated profile is converted to a flat dictionary and loaded as initial facts into the inference engine's working memory.

Step 5 Rule Evaluation

The engine iterates over the rule base. Conditions are tested against working memory; matching rules are collected into a conflict set.

Step 6 Conflict Resolution

The conflict set is sorted by rule priority and certainty factor. The highest-ranked rule fires first, adding its conclusions to working memory.

Step 7 Trace Recording

Each rule firing is appended to the reasoning trace, capturing which rule fired, what conclusions were drawn, and with what certainty.

Step 8 Output Processing

The RecommendationProcessor computes the final risk score and structures the recommendations, strengths, and risk factors.

Step 9 API Response

The Flask route assembles the final JSON response and returns it to the caller with an HTTP 200 status code.

5. Expert System Design

5.1 Inference Strategy

The system uses forward chaining (data-driven reasoning). Starting from the known facts in the student profile, the engine repeatedly scans the rule base and fires every rule whose conditions are satisfied by the current working memory. This continues until no new rules fire or the maximum iteration limit is reached.

Forward chaining is appropriate for this domain because the initial facts (the student profile) are fully known at the start of each session, and the goal is to derive as many relevant conclusions as possible from those facts rather than to prove a specific hypothesis.

5.2 Certainty Factor Reasoning

Each rule carries a certainty factor (CF) in the range [0, 1] that quantifies the confidence of its conclusion given that the rule's conditions are met. The inference engine uses a combination function to aggregate evidence from multiple rules supporting the same conclusion, ensuring that corroborating evidence increases confidence while conflicting evidence reduces it. The combined certainty propagates into the final risk score.

5.3 Conflict Resolution

When multiple rules are eligible to fire simultaneously, the engine applies conflict resolution to select the rule to fire next. The resolution strategy prioritises rules in the following order:

- Primary: Rule priority (higher priority fires first)
- Secondary: Certainty factor (higher CF breaks ties)

This strategy ensures that high-confidence, high-importance rules always take precedence, producing consistent and predictable reasoning behaviour.

5.4 Rule Structure

Every rule in the knowledge base has the following attributes:

- name — unique identifier for the rule
- conditions — list of predicates tested against working memory
- actions — list of conclusions added to working memory on firing
- cf — certainty factor expressing rule confidence (0 – 1)
- priority — integer controlling firing order
- description — plain-language explanation for trace readability

5.5 Rule Categories

The knowledge base is organised into the five categories described in the table below. Categories are processed in priority order; composite rules that require earlier conclusions to be present in working memory are assigned lower priority than their prerequisite rules.

Rule Category	Focus Area
Attendance Rules	Detects problematic or borderline attendance patterns
Academic Performance Rules	Evaluates grades, GPA trends, and subject-level risk
Behavioral Risk Rules	Identifies stress, engagement, and conduct indicators
Composite / Conflict Rules	Combines multiple factors; resolves contradictory evidence
Recommendation Rules	Generates targeted support and intervention suggestions

6. Input Validation

All incoming student profiles are validated by input_handler/input_validator.py before inference begins. Validation is split into two severity levels:

- Hard errors — processing is halted and a 400 Bad Request is returned. Examples include missing required fields or scores outside the 0–100 range.
- Warnings — processing continues but the warning is returned in the API response. Examples include unusual but technically valid values such as zero study hours.

Field	Constraint	Notes
Scores & Percentages	0 – 100	Hard error if outside range
Study Hours	0 – 18	Warning raised for zero hours
Stress Level	1 – 5	Integer scale; hard error otherwise
Trend Values	improving / stable / declining	Must match allowed values exactly
Required Fields	Present	Hard error if any required field missing

The table above summarises all validated fields. Any field not listed above is accepted as-is without transformation.

7. API Reference

The Flask backend exposes three REST endpoints. All request and response bodies use the application/json content type. The base URL for the API in local development is http://localhost:5000.

Method	Endpoint	Description
GET	/api/health	Returns service status and version information
GET	/api/samples	Returns 10 pre-built student profiles for testing and demo
POST	/api/analyze	Accepts JSON student profile; returns risk score, verdict, strengths, risk factors, recommendations, and reasoning trace

7.1 GET /api/health

Returns the current health status and version of the API service. No request body is required.

Example response:

```
{ "status": "ok", "version": "1.0.0" }
```

7.2 GET /api/samples

Returns an array of 10 pre-built student profile objects suitable for UI demonstrations and API integration testing. Each profile object conforms to the same schema as the POST /api/analyze request body.

7.3 POST /api/analyze

Accepts a JSON student profile and returns a complete risk assessment. The response object contains the following fields:

- verdict — overall risk classification (e.g., High Risk, At Risk, Moderate Risk, Low Risk)
- risk_score — numeric composite risk score (0 – 100)
- strengths — list of identified positive academic factors
- risk_factors — ranked list of detected risk signals
- recommendations — prioritised list of intervention suggestions
- reasoning_trace — ordered list of rules that fired and their conclusions
- warnings — any non-fatal validation warnings raised during processing

8. Project Structure

The project repository is organised as follows. Each directory is self-contained and has a well-defined role in the system:

`api/`	Flask application factory, route blueprints, and server configuration
`api/app.py`	Main Flask app entry point and server startup
`api/routes/`	Blueprint modules for each API endpoint group
`api/config.py`	Environment variables and Flask runtime configuration
`frontend/`	React application source, Vite configuration, and package manifest
`inference_engine/`	Core inference engine, working memory, and conflict resolver
`inference_engine/engine.py`	Rule evaluation loop, fact store, and trace recorder
`knowledge_base/`	Expert rule definitions and knowledge base builder
`knowledge_base/rules.py`	All rule objects and the KB construction function
`input_handler/`	Student profile model and validation module
`input_handler/student_profile.py`	StudentProfile dataclass definition
`input_handler/input_validator.py`	Field validation logic (errors + warnings)
`output_handler/`	Result formatting and report generation
`output_handler/recommendation.py`	Risk scoring, strengths, and recommendation processor
`output_handler/report_generator.py`	Console and UI-formatted report builder
`tests/`	Automated test scenarios and runner
`tests/test_scenarios.py`	10 student scenarios with assertions and result summaries
`ui/`	Optional Streamlit interface
`ui/app.py`	Streamlit app entry point
`main.py`	CLI entry point supporting flask, demo, and test sub-commands

9. Deployment & Running the System

9.1 Prerequisites

- Python 3.8 or higher with pip
- Node.js 16 or higher with npm
- All Python dependencies installed from requirements.txt
- Node dependencies installed within frontend/ via npm install

9.2 Starting the Backend

From the repository root, start the Flask development server using either of the following commands:

```
python api/app.py
```

Alternatively, using the main.py helper:

```
python main.py flask
```

The server will start on http://localhost:5000 by default. All three API endpoints will be immediately available.

9.3 Starting the Frontend

Navigate to the frontend/ directory and run the following commands:

```
npm install
npm run dev
```

The React development server will start on http://localhost:5173 by default and will hot-reload on file changes.

9.4 Running the Terminal Demo

To explore the system's output without starting the frontend, run the demo mode from the repository root:

```
python main.py demo
```

This command loads the knowledge base, runs an analysis on a built-in sample profile, and prints a formatted report to the console.

9.5 Running the Test Suite

Execute all ten student scenario tests with the following command:

```
python main.py test
```

The test runner evaluates each scenario against expected outputs, prints a summary table, and exits with a non-zero code if any assertions fail.

10. Testing Strategy

The test suite in tests/test_scenarios.py covers ten diverse student profiles designed to exercise different rule paths and risk levels across the knowledge base. Each scenario includes:

- A fully populated StudentProfile object with realistic field values
- Assertions on the expected risk verdict (e.g., High Risk, Low Risk)
- Assertions on the presence or absence of specific recommendations
- A printed summary of which rules fired and the resulting conclusions

The scenarios are designed to collectively achieve high coverage of all five rule categories, ensuring that changes to the knowledge base are immediately surfaced as test failures.

10.1 Scenario Coverage

- High-risk profile: poor attendance, declining grades, high stress
- Low-risk profile: excellent attendance, strong GPA, improving trend
- Borderline profile: moderate attendance, mixed academic signals
- Behavioural risk profile: engagement issues despite adequate grades
- Recovering profile: improving trend from a previously poor baseline
- And five additional scenarios covering edge cases in the validator and rule interactions

10.2 Recommended Additional Tests

The following types of automated tests are recommended as next steps to increase overall system confidence:

- Unit tests for individual validator functions in input_handler/input_validator.py
- Unit tests for each rule in the knowledge base, fired in isolation
- Unit tests for the RecommendationProcessor risk scoring algorithm
- Integration tests covering the full POST /api/analyze pipeline
- Frontend component tests using React Testing Library

11. Known Gaps & Limitations

The following limitations are acknowledged in the current version of the system:

- Environment dependency — the system requires both a correctly configured Python environment and a Node.js installation. No containerised deployment is provided in this release.
- No persistent storage — student profiles and analysis results are not persisted between sessions. Each API call is stateless.
- Advisory only — the system is designed to support, not replace, professional academic counselling. Its recommendations should be reviewed by a qualified person before action is taken.
- Rule explainability — while a reasoning trace is provided, the trace currently lacks detailed metadata such as which specific fact values triggered each condition. Richer trace metadata would improve transparency.
- No historical tracking — the system does not maintain longitudinal data and therefore cannot detect trends across multiple assessment sessions for the same student.
- frontend/node_modules/ — this directory must be generated locally and is excluded from version control via .gitignore.

12. Future Roadmap

The following improvements are recommended for future releases, ordered by priority:

12.1 Short Term

- Add Docker Compose configuration for one-command deployment of backend and frontend
- Implement unit and integration tests for the validator, engine, and recommendation processor
- Enhance the reasoning trace with per-condition match details for improved explainability

12.2 Medium Term

- Expand the frontend to visualise risk scores with charts and trend indicators
- Add student session history so that multiple assessments can be compared over time
- Introduce an authenticated user model for institutional deployments

12.3 Long Term

- Implement a machine-learning layer to learn rule weights from historical outcome data
- Build an administrative dashboard for cohort-level risk monitoring
- Develop an LMS plugin (Canvas, Moodle) to ingest student data automatically
- Support multi-language interfaces for international institutional adoption

13. Glossary

Term	Definition
Certainty Factor (CF)	A numeric value in [0, 1] attached to each rule that quantifies confidence in the rule's conclusion when its conditions are met.
Conflict Resolution	The strategy used to select which rule fires when multiple rules are eligible simultaneously. The system uses priority then CF as tiebreaker.
Expert System	A computer system that emulates the decision-making ability of a human expert through a knowledge base and an inference engine.
Forward Chaining	An inference strategy that starts from known facts and applies rules repeatedly until no new conclusions can be drawn.
Inference Engine	The algorithmic component that applies rules from the knowledge base to facts in working memory to derive conclusions.
Knowledge Base	The structured collection of domain-specific rules and facts that encode expert knowledge about academic risk.
Risk Score	A composite numeric score (0 – 100) computed from the weighted conclusions of the inference engine, where higher scores indicate greater risk.
StudentProfile	The data model representing a single student's academic and behavioural attributes used as input to the inference engine.
Working Memory	The dynamic fact store used by the inference engine. It is initialised with the student profile and grows as rules fire and add conclusions.